Program Specification: Semantic Binding Structured-CSV-to-Flat-File Conversion

1. Purpose

This document specifies the semantic binding conversion program used by the UADC Proof of Concept.

The program converts a Phase 1 UADC Structured CSV into a Phase 2 target flat file by applying a semantic binding CSV. The target flat file can be pipe-separated PSV or comma-separated CSV. The ADS PSV files and ISO 21378 ADC invoice CSV files use the same processing model.

Implementation-level processing details are also summarized in **../README_SCRIPT_PROCESSING.md**.

All paths in this document are relative to the **UADC_PoC** working directory.

2. Main Program

Program:

```
src/semantic_binding.py
```

The converter is independent of XML syntax. It reads Structured CSV rows and maps values to target columns by **semantic_path**.

3. Input Files

3.1 Structured CSV input

The input is a Phase 1 Structured CSV file generated by **src/syntax_binding.py**.

Example:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

The input may also be a directory containing multiple Structured CSV files. In that case, each **.csv** file is converted separately.

3.2 Semantic binding CSV

Semantic binding CSV files are under:

```
specs/bindings/semantic/
```

Current ADS files include:

```
ADS_Invoices_Received_PSV_Binding.csv
ADS_Invoices_Generated_PSV_Binding.csv
ADS_Invoices_Received_Lines_PSV_Binding.csv
ADS_Invoices_Generated_Lines_PSV_Binding.csv
ADS_Supplier_Listing_PSV_Binding.csv
ADS_Customer_Master_PSV_Binding.csv
```

Current ISO 21378:2019 ADC invoice files include:

```
ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv
```

They implement the flat invoice views from Tables 38, 39, 53, and 54. Repeated TAX groups are expanded to **Tax1** through **Tax4** columns. Repeated BUSINESS SEGMENT values are represented by **Business_Segment_01** through **Business_Segment_05** columns. This physical expansion is needed because a CSV header cannot contain repeated field names.

4. Binding Table Contract

The semantic binding table starts from the target definition table and adds UADC mapping columns:

```
field_no,field_name,level,flat_file_data_type,length,description,source_do
cument,semantic_path,type,multiplicity,mapping_status,mapping_note
```

4.1 A rows

Rows with **type=A** emit target columns.

Important fields:

- **field_no** controls output column order.
- **field_name** is the emitted target column name.
- **semantic_path** identifies the UADC Structured CSV value.
- **description** comes from the target definition document.
- **mapping_status** is **direct**, **approximate**, **requires_transformation**, or **not_available**.
- **mapping_note** explains a semantic approximation or data gap.

The converter derives the Structured CSV source column from the final segment of **semantic_path**.

Example:

```
$.invoice.invoiceNumber -> InvoiceNumber
$.invoice.seller.sellerIdentifier -> SellerIdentifier
```

4.2 C rows

Rows with **type=C** define UADC semantic classes used for row-scope resolution.

Important fields:

- **field_name** is the semantic class name.
- **description** is copied from the Structured CSV model definition.
- **semantic_path** identifies the semantic class.
- **multiplicity** determines whether the class can repeat.

Repeated classes are those whose multiplicity upper bound is *****, **n**, **unbounded**, or greater than 1.

The converter derives the Structured CSV dimension column from the class semantic path.

Example:

```
$.invoice.invoiceLine -> dInvoiceLine
```

5. Repeated Row Scope

The converter determines the target row scope from the deepest repeated **C** ancestor used by non-indexed **A** rows.

Invoice-level targets have no repeated row scope. The converter merges sparse Structured CSV rows into one target row per invoice.

Line-level targets use **$.invoice.invoiceLine** as row scope. The converter emits one target row for each **dInvoiceLine** occurrence and merges parent invoice values into each line row.

Other repeated classes, such as VAT breakdown or allowances, follow the same rule when a semantic binding table uses those classes as non-indexed repeated ancestors.

6. Indexed Repeated Values

Use zero-based indexed semantic paths when a repeated group must be expanded horizontally into target columns.

Example:

```
$.invoice.vatBreakdown[0].vatCategoryCode
$.invoice.vatBreakdown[1].vatCategoryCode
$.invoice.vatBreakdown[2].vatCategoryCode
```

Indexed paths do not create additional output rows. They select the specified occurrence inside the current invoice or current repeated row context.

The ISO 21378 header bindings use indexed VAT breakdown paths for **Tax1** through **Tax4**. Invoice notes and payment instructions also use occurrence zero because their ADC header fields are singular. The detail bindings use the repeated **InvoiceLine** class as the output row scope. EN 16931 permits one VAT classification per invoice line, so only **Tax1_Type_Code** can be mapped directly on detail rows.

7. Internal Data Structures

read_csv_rows(path) returns:

```
list[dict[str, str]]
```

Each dictionary is one CSV row keyed by header name.

SemanticClass stores:

- **semantic_path**;
- **multiplicity**.

Repeated **SemanticClass** rows are collected as:

```
dict[str, SemanticClass]
```

SemanticBinding stores one target-column binding:

- **sequence**;
- **target_column**;
- **semantic_path**;
- **normalized_semantic_path**;
- **source_column**;
- **repeat_group_path**;
- **repeat_group_column**;
- **repeat_index**;
- **default_value**;
- **required**.

load_bindings(binding_csv) returns:

```
list[SemanticBinding]
```

The list is sorted by **field_no** or **sequence**, then by target column name.

8. Processing Functions

load_bindings(binding_csv) reads the binding table, records repeated **C** classes, creates **SemanticBinding** objects for **A** rows, derives source columns from semantic paths, and resolves repeated row scopes from the binding table.

row_scope_group(bindings) selects the deepest repeated group used by the target view. This determines whether the output is invoice-level or repeated-row-level.

transform_rows(rows, bindings) applies the binding table to Structured CSV rows. If no repeated row scope exists, it merges sparse rows into invoice-level target rows.

transform_repeated_group_rows(rows, bindings, scope_path, scope_column) emits one target row per repeated group occurrence, such as one row per **dInvoiceLine**.

row_repeat_indices(source_row, bindings, repeat_counts) assigns zero-based occurrence numbers for indexed repeated paths.

merge_values(target_row, source_row, bindings, repeat_indices) copies non-empty Structured CSV values into target columns. It uses the first non-empty value for each target column.

output_path(input_csv, output_dir, extension, output_filename, binding_csv) writes output under:

```
out/phase2/<target-family>/<structured-csv-stem>/<target-view>.<extension>
```

unless **--output-filename** is supplied for a single input file.

9. Output Contract

The output header follows the order of **A** rows in the semantic binding CSV.

Default output presets:

Format	Delimiter	Extension
psv	**	**
csv	,	**.csv**

The target view stem is derived from the binding file name. For example:

```
ADS_Invoices_Received_PSV_Binding.csv -> Invoices_Received.psv
ADS_Customer_Master_PSV_Binding.csv -> Customer_Master.psv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv -> PUR_Invoice_Received.csv
```

10. Non-Goals

- The converter does not parse XML.
- The converter does not use XPath.
- The converter does not infer mappings from target column names alone.
- The converter does not read an external LHM CSV at runtime.
- The converter does not generate xBRL-CSV metadata JSON.
- The converter does not derive fiscal year or accounting period without an accounting calendar.
- The converter does not calculate line gross amounts or line tax amounts.
- The converter does not invent ERP audit trail, posting account, status, or additional business segment values that are absent from EN 16931.